

Séance 4 – Faire se comporter `IntArray` comme un type natif

Copie • déplacement • opérateurs

Stéphane Abide

Université Côte d’Azur – LJAD

Cours : 1 h 30 – TD/TP : 2 heures

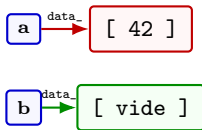
Copier : comportement attendu et copie implicite

Fin de la séance 3 : copie interdite avec = **delete**. Ici, nous la réactivons provisoirement pour observer la copie générée par le compilateur.

`std::vector<int>`

```
std::vector<int> a{42};  
  
std::vector<int> b{a};  
b.clear();
```

Copie de vector

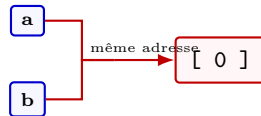


Deux objets, deux allocations.
a contient encore 42; b est vide.

IntArray actuel

```
IntArray a{8};  
a.push_back(42);  
  
IntArray b{a}; // copie reactivee  
b.fill(0);
```

Copie implicite d'IntArray



Deux objets, une allocation.
fill(0) change les données vues par a et b.

`IntArray b{a}` copie actuellement l'adresse de `data_`, pas le tableau.

La copie implicite copie le pointeur

```
IntArray a{8};  
a.push_back(42);  
IntArray b{a};  
b.fill(0);
```

La fonction générée par le compilateur recopie chaque membre :

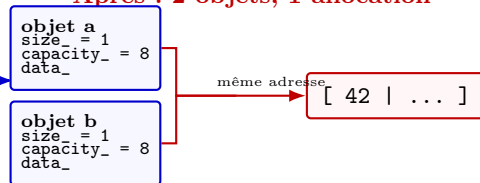
size_ capacity_ data_

Pour data_, la valeur recopiée est une **adresse**.

Avant : 1 objet, 1 allocation



Après : 2 objets, 1 allocation



Modification partagée

b.fill(0) modifie aussi la donnée vue par a.

Destruction incorrecte

Deux destructeurs appellent delete[] sur la même adresse.

Copier data_, c'est copier une adresse ; copier IntArray doit créer une **autre allocation**.

Construire ou affecter : deux situations

Construire un objet

```
IntArray b{a};
```

Avant
b n'existe pas

construire →

Après
nouvel objet b

constructeur de copie

Remplacer un état

```
b = a;
```

Avant
b existe déjà

remplacer →

Après
le même b

affectation par copie

Construire : la destination n'existe pas encore. **Affecter** : elle existe déjà et doit être remplacée.

La même distinction reviendra pour le déplacement.

La règle des cinq dans l'en-tête de IntArray

int_array.hpp

```
class IntArray {
public:
    ~IntArray();

    IntArray(const IntArray& other);
    IntArray& operator=(const IntArray& other);

    IntArray(IntArray&& other) noexcept;
    IntArray& operator=(IntArray&& other) noexcept;

    // interface ordinaire...
};
```

Destruction : 1

libérer la ressource à la fin de vie

Copie : 2

construire ou affecter
en dupliquant la ressource

Déplacement : 2

construire ou affecter
en transférant la ressource

$$1 + 2 + 2 = 5$$

Lorsqu'une classe possède directement une ressource, destruction, copie et déplacement doivent être pensés ensemble : c'est la **règle des cinq**.

Elle ne demande pas d'écrire cinq fonctions dans toutes les classes.

Constructeur de copie : créer b à partir de a

Appel

```
IntArray b{a};
```

b n'existe pas encore.

a est la source, non modifiée.

Signature

```
IntArray(  
    const IntArray& other);
```

other désigne a.

objet a
size_ = 3
capacity_ = 8
data_

allocation A : [4 | 7 | 2]

objet b
size_ = 3
capacity_ = 8
data_

allocation B : [4 | 7 | 2]

nouvelle allocation

$a.data_ \neq b.data_$

$a.size() = b.size()$

contenu de a = contenu de b

Contrat du constructeur de copie

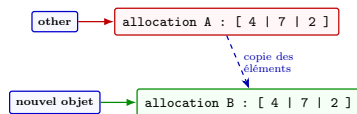
Même contenu initial, mais ressource indépendante.

Constructeur de copie : dupliquer la ressource

int_array.cpp

```
IntArray::IntArray(const IntArray& other)
: capacity_{other.capacity_},
  size_{other.size_},
  data_{capacity_ == 0
        ? nullptr
        : new int[capacity_]}
{
    if (size_ != 0) {
        std::copy_n(other.data_, size_, data_);
    }
}
```

Effet en mémoire



Taille et capacité sont recopiées;
new crée l'allocation B.

Coût $\mathcal{O}(n)$

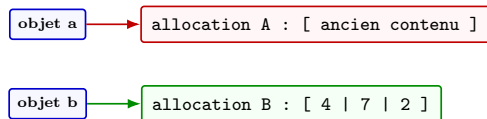
new crée une autre allocation; **std::copy_n** y recopie les éléments : les deux objets sont indépendants.

Affectation par copie : remplacer une ressource existante

```
a = b;  
// équivaut à l'appel :  
a.operator=(b);
```

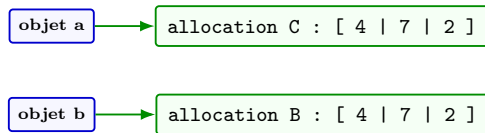
*this désigne a, la **destination** ; other désigne b, la **source**.

Avant l'affectation



a et b possèdent deux contenus différents.

Après l'affectation



L'allocation A est libérée ; b reste inchangé.

a reste le même objet, mais son ancienne ressource est remplacée par une **copie indépendante** de celle de b.

Affectation par copie : une implémentation sûre

Ordre sûr

- ① créer ;
- ② copier ;
- ③ libérer l'ancien ;
- ④ installer le nouveau.

Auto-affectation

`a = a` doit rester sans effet.

`return *this;` renvoie l'objet appelant, donc `a`.

```
IntArray& IntArray::operator=(const IntArray& other)
{
    if (this == &other) {
        return *this;
    }
    int* new_data{other.capacity_ == 0
        ? nullptr : new int[other.capacity_]};
    if (other.size_ != 0) {
        std::copy_n(other.data_, other.size_, new_data);
    }
    delete[] data_;           // apres reussite
    data_ = new_data;
    capacity_ = other.capacity_;
    size_ = other.size_;
    return *this;
}
```

Il faut préparer le remplacement avant de libérer l'ancienne ressource.

Constructeur de déplacement : transférer la ressource

Appel

```
IntArray b{std::move(a)};
```

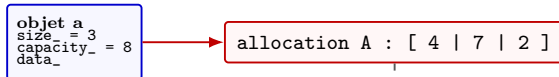
b n'existe pas encore.
a est l'objet source.

Signature

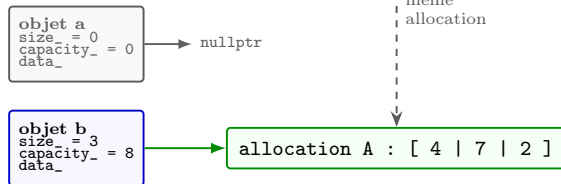
```
IntArray(IntArray&& other)  
    noexcept;
```

other désigne a.

Avant



Après



aucune allocation nouvelle aucun élément recopié
quelques champs transférés $O(1)$

Le tableau ne bouge pas : sa propriété passe de a à b ; a reste vide et valide.

Constructeur de déplacement : créer b en récupérant a

Appel

```
IntArray b{std::move(a)};
```

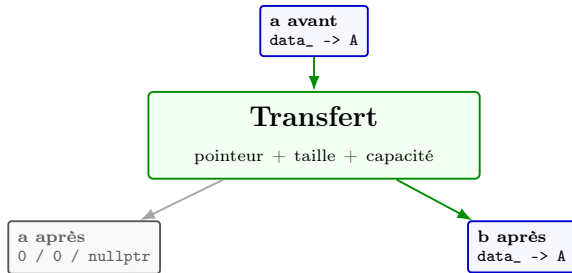
b n'existe pas encore.
Il récupère la ressource de a.

Signature

```
IntArray(IntArray&& other)  
noexcept;
```

other désigne l'objet source a.

Réutiliser la source vide
Notre contrat autorise ensuite
a.push_back(3).



```
const std::size_t new_capacity{  
    capacity_ == 0 ? 1 : 2 * capacity_};
```

Le constructeur de déplacement crée la destination
par transfert, puis laisse la source dans un état valide.

std::exchange : prendre puis remplacer

D'abord, l'idée

```
data_ = other.data_;  
other.data_ = nullptr;
```

- ① récupérer l'adresse ;
- ② enlever cette adresse à `other`.

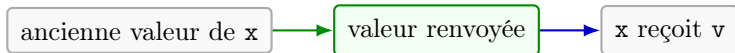
Même mécanisme pour `size_` et `capacity_`.

Le raccourci

```
data_{std::exchange(  
    other.data_, nullptr)}
```

`std::exchange(x, v)`

```
ancienne = x;  
x = v;  
return ancienne;
```



`std::exchange` prend une ancienne valeur puis la remplace.

Dans le déplacement, ce n'est pas un échange : la destination « vole » la ressource de la source.

Implémenter le constructeur de déplacement

```
IntArray::IntArray(IntArray&& other) noexcept
: capacity_{std::exchange(other.capacity_, 0)},
  size_{std::exchange(other.size_, 0)},
  data_{std::exchange(other.data_, nullptr)}
{}
```

1 capacity_

prend l'ancienne capacité;
other.capacity_ devient 0.

2 size_

prend l'ancienne taille;
other.size_ devient 0.

3 data_

prend l'ancien pointeur;
other.data_ devient nullptr.

noexcept : cette opération ne doit pas lever d'exception.

Aucune allocation, aucune copie des éléments : la source devient 0 / 0 / nullptr.

Affectation par déplacement : `b = std::move(a)`

```
b = std::move(a);  
// équivaut à l'appel :  
b.operator=(std::move(a));
```

Rôles dans l'appel

`*this` désigne `b` : destination

`other` désigne `a` : source

`b` existe déjà et possède peut-être une ressource.

b existe déjà

[ancienne ressource]

a source

[ressource de a]

La destination existe déjà : elle doit libérer son ancienne ressource avant de récupérer celle de la source.

Implémenter l'affectation par déplacement

```
IntArray& IntArray::operator=(IntArray&& other) noexcept
{
    if (this != &other) {
        delete[] data_; // ressource de *this
        capacity_ = std::exchange(other.capacity_, 0);
        size_ = std::exchange(other.size_, 0);
        data_ = std::exchange(other.data_, nullptr);
    }
    return *this;
}
```

1 delete[] data_

libère l'ancienne ressource de
la destination.

2 std::exchange

transfère capacité, taille et
pointeur depuis la source.

3 other

devient vide, valide et
destructible : 0 / 0 /
nullptr.

if (this != &other) protège l'auto-affectation; return *this; renvoie la destination, ici b.

Affecter par déplacement : libérer l'ancienne
ressource, puis récupérer celle de la source.

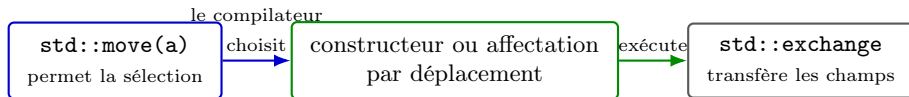
std::move(a) ne déplace rien

Construire b

```
IntArray b{std::move(a)};  
// sélectionne :  
IntArray::IntArray(IntArray&&);
```

Remplacer b

```
b = std::move(a);  
// sélectionne :  
IntArray::operator=(IntArray&&);
```



std::move

ne recopie rien, ne vide pas **a** et ne modifie aucun de ses champs.

Opération de déplacement

réalise le transfert ; dans notre code, elle utilise **std::exchange**.

std::move permet de sélectionner le déplacement ; le constructeur ou l'opérateur réalise réellement le transfert.

Règle des cinq pour comprendre, règle de zéro pour produire

Gestion directe

```
std::size_t capacity_;  
std::size_t size_;  
int* data_;
```



destructeur
constructeur de copie
affectation par copie
constructeur de déplacement
affectation par déplacement

5 opérations

Composition

```
std::vector<int> data_;  
data_.size();  
data_.capacity();
```



vector sait déjà :
détruire
copier
déplacer

aucune opération spéciale

0 opération

Message pratique

Pour comprendre : règle des cinq.

Pour produire : règle de zéro.

Une interface naturelle pour un type utilisateur

`a[i]`

accéder à un élément

`int&` si `a` est modifiable

`const int&` si `a` est
constant

`a += b`

modifier `a` sur place

renvoie `*this`
pour respecter l'usage
habituel

`c = a + b`

produire une nouvelle valeur

`a` et `b` restent inchangés

Principe

Un opérateur surchargé reste une fonction, mais doit conserver le sens attendu de son symbole.

`operator+` peut travailler sur une copie de son premier opérande précisément parce que le **constructeur de copie** est correct.

Le détail de `+=` et `+` reste un approfondissement ; le TD obligatoire se concentre sur `operator[]`.

Séance 4 – Les cinq opérations autour d'une ressource

**Constructeur
de copie**

créer et dupliquer

**Affectation
par copie**

préparer et remplacer

**Constructeur
de déplacement**

créer et transférer

**Affectation
par déplacement**

libérer et transférer

Destructeur libérer la ressource possédée

Copie même contenu, mémoire
différente – $\mathcal{O}(n)$

Déplacement même allocation,
nouveau propriétaire – $\mathcal{O}(1)$

Règle des cinq pour comprendre.

Règle de zéro pour produire.

Comprendre qui possède la ressource, quand elle est dupliquée et
quand elle est transférée.